

东莞理工学院

本科毕业设计

毕业设计题目： 基于 Playwright 的浏览器 RPA 画
布工作流系统的设计与开发

学生姓名：

学 号：

学 院： 计算机科学与技术学院(网络空间
安全学院)

专业班级：

指导教师姓名及职称： ---副教授

起止时间： 2025 年 12 月—— 2026 年 5 月

摘要

网页表单录入、信息采集、状态监控等重复性浏览器操作在企业数字化转型中日益普遍。传统人工处理方式耗时长、易出错，难以适应高频、规则明确的业务场景；而商业化 RPA 产品通常价格较高、配置复杂，对中小企业和普通业务人员并不友好。针对上述问题，本文设计并实现了基于 Playwright 的浏览器 RPA 画布工作流系统 TheTower，旨在通过低代码、可视化的方式降低浏览器自动化技术的使用门槛。

技术上，系统采用前后端分离架构。前端基于 React、Umi 与 ReactFlow 构建可视化编排界面，支持节点拖拽、连线、参数配置等编辑能力；后端基于 Ktor 与 Playwright Java 构建执行与管理服务，负责模板持久化、运行调度、步骤解析、浏览器自动化执行及事件推送。前后端之间通过 REST 接口完成数据交换，通过 WebSocket 实时推送运行状态和调试事件，实现流程执行过程的可视化监控。

围绕“可编排、可执行、可调试、可扩展”的设计目标，系统在 0.1.0 版本中完成了编辑器体验完善、高级调试、模板市场、调度任务等首版能力闭环。其中，执行链路支持多种节点类型；调试链路支持条件断点、时间旅行、远程操控和上下文查看；产品化能力支持模板分享、市场复用、多工作空间隔离和在线协作。

经测试验证，后端单元测试与接口烟雾测试、前端回归测试、真实后端联调测试均已通过，模板管理、运行调试、数据文件节点、网络监听、市场、调度、认证和协作等主线功能完成验证。画布端已完成 50 节点性能测量与回归资产沉淀，后端执行链路完成多浏览器兼容验证。本文的研究与实现为低代码浏览器自动化系统的设计提供了可落地的工程实践，也为后续协作冲突自动合并、细粒度权限和更强持久化能力的演进奠定了基础。

关键词 浏览器 RPA、可视化流程编排、Playwright、ReactFlow、低代码

Abstract

With the digital transformation of enterprises and the improvement of personal office efficiency, repetitive browser operations such as web form entry, information collection, and status monitoring have become increasingly common. The traditional manual processing method is time-consuming and prone to errors, making it difficult to adapt to high-frequency and clearly-defined business scenarios; while commercial robot process automation products are usually expensive and complex in configuration, and are not friendly to small and medium-sized enterprises and ordinary business personnel. To address these issues, this paper designs and implements a browser RPA canvas workflow system called TheTower, aiming to lower the threshold for using browser automation technology through low-code and visual methods.

This system adopts a front-end and back-end separation architecture. The front-end is built using React, Umi, and ReactFlow to create a visual process orchestration interface, supporting node dragging, connection, parameter configuration, and other editing capabilities; the back-end is built using Ktor and Playwright Java to handle template persistence, run scheduling, step parsing, browser automation execution, and event pushing. The system completes data exchange through REST interfaces and real-time pushes the running status and debugging events through WebSocket to achieve visual monitoring of the process execution.

Centering on the design goals of "reconfigurable, executable, debuggable, and scalable", the system completed the first version capability loop of editor experience improvement, advanced debugging, template market, and scheduling tasks in version 0.1.0. Among them, the execution link supports a series of various node

types; the debugging link supports conditional breakpoints, time travel, remote control, and context viewing; the productization capability supports template sharing, market reuse, multi-workspace isolation, and online collaboration.

The test results show that the system has completed backend unit testing, interface smoke testing, front-end regression testing, and real backend integration testing. Both backend compilation and front-end compilation passed, and the template management, run debugging, data file nodes, network monitoring, market, scheduling, authentication, and collaboration mainlines were all verified. The system has completed 50 node canvas performance measurement and regression asset accumulation, and the backend execution link completed multi-browser compatibility verification. The research and implementation of this paper provide a practical engineering practice for the design of low-code browser automation systems, and lay the foundation for the subsequent evolution of automatic conflict merging, fine-grained permissions, and stronger persistence capabilities.

Keywords browser RPA, visual workflow orchestration, Playwright, ReactFlow, low-code

目 录

第 1 章 绪论	7
1.1 课题应用背景	7
1.2 现有产品和存在问题	7
1.3 论文的主要工作	8
1.4 章节安排	8
第 2 章 相关技术和开发环境	10
2.1 相关技术/相关研究调研	10
2.2 相关技术 2	10
2.3 相关技术 3	11
2.4 开发环境	11
2.4.1 软件环境	11
2.4.2 硬件环境	12
第 3 章 系统设计	13
3.1 可行性研究	13
3.1.1 经济可行性	13
3.1.2 技术可行性	13
3.1.3 运行可行性	13
3.2 业务需求分析	14
3.3 功能需求分析	14
第 4 章 概要设计	15
4.1 系统设计	15
4.1.1 系统架构设计	15
4.1.2 系统功能模块	15
4.2 系统核心功能设计	16
4.2.1 关键概念定义	16
4.2.2 模型建立	16
4.3 系统接口设计	16
第 5 章 详细设计	18

5.1 流程编排模型与前端交互设计	18
5.2 后端执行引擎设计与实现	18
5.3 调试机制与运行时上下文设计	19
5.4 协议、协作与数据持久化设计	19
第 6 章 系统实现	21
6.1 模板管理与编辑器实现	21
6.2 执行与调试功能实现	21
6.3 数据与网络监听节点实现	21
第 7 章 系统测试	23
7.1 测试环境和测试方法	23
7.2 测试用例设计和测试方案	23
7.3 结果和分析	24
第 8 章 总结与展望	25
8.1 总结	25
8.2 展望	25
参考文献	27
致谢	28

第 1 章 绪论

1.1 课题应用背景

随着数字化办公的推进，大量业务流程已迁移至浏览器端，例如后台信息录入、订单状态查询、运营数据采集、内容审核和页面监控等。这类任务通常具备规则明确、重复频率高、人工价值密度低等特征。若完全依赖人工处理，不仅效率受限，而且在长时间重复操作下容易产生操作失误、信息遗漏和数据不一致等问题。对于中小企业和非技术部门而言，这类工作虽然适合自动化，但现有解决方案往往存在成本过高或技术门槛过高等问题，难以满足实际需求。

传统浏览器自动化通常依赖脚本开发完成。尽管 Selenium、Playwright 等自动化框架已较为成熟，但其使用前提仍是操作者具备编程能力，需要理解浏览器元素定位、等待机制、流程控制、异常处理和数据持久化等技术细节。另一方面，商业化 RPA 平台虽提供了图形化编排界面，却普遍存在授权成本较高、场景适配封闭、定制能力受限等问题，难以满足毕业设计等需要灵活调整的场景需求。

TheTower 即是这一思路的具体实践——一个基于 Playwright 的浏览器 RPA 画布 workflow 系统。用户可在画布上拖拽节点、配置参数、连线编排 workflow；后端负责将流程定义解析为具体的执行步骤，驱动真实浏览器运行；同时运行状态、调试信息和错误日志实时回传至前端，用户可直观查看执行进度并定位问题。这种设计保留了自动化框架的执行能力，同时通过可视化编排降低了使用门槛。用户无需编程基础，只需理解业务流程即可搭建可用的自动化流程。

1.2 现有产品和存在问题

现有网页自动化解决方案大致可分为两类。第一类为代码驱动型方案，例如直接使用 Playwright、Selenium 或 Puppeteer 编写自动化脚本。这类方案灵活度高、可控性强，适用于复杂场景和深度定制，但开发与维护成本较高，普通业务人员难以直接上手。第二类为可视化 RPA 平台，通过拖拽方式降低使用门槛，适用于标准化业务流程，但通常更偏向企业级交付，其部署、授权、插件体系和定制成本较高，对于轻量化教学项目和中小规模业务并不完全匹配。

综合来看，现有产品在落地应用层面存在四方面局限。最突出的是技术门槛偏高。脚本式自动化方案要求使用者具备编程基础，需掌握选择器规则、调试方法及工程化实践等技能，对非技术人员形成显著障碍。其次是总体拥有成本较高。成熟的商业化平台多采用付费授权与企业级部署模式，其费用结构往往超出小团队及个人用户的可承受范围。再者，执行过程缺乏透明度。许多轻量级脚本方案虽可实现基本运行，但难以向非技术用户提供清晰的运行状态反馈、失败步骤定

位、变量上下文展示及错误日志追溯，导致问题排查困难。此外，扩展能力也存在割裂。部分平台在文件读写、网络监听、调试支持、协作机制及模板复用等关键能力上支持有限，难以形成完整的闭环体验。

针对上述局限，TheTower 选取"可视化编排+浏览器自动化引擎+实时状态反馈+产品化首版能力"作为核心路线。具体而言，系统前端以 ReactFlow 构建节点式可视化画布，为用户提供拖拽式工作流设计体验；后端则以 Playwright 执行器承载真实浏览器操作，并通过 REST 与 WebSocket 形成统一的运行与调试协议。与纯脚本方案相比，TheTower 更强调可视化、可观测性与可复用性；与重量级商业平台相比，TheTower 更强调轻量、本地可部署性与工程可控性。

1.3 论文的主要工作

本文围绕浏览器 RPA 画布工作流系统的设计与实现，完成以下六方面工作。

在需求分析与架构设计方面，基于重复网页任务的自动化场景，明确系统应支持可视化流程编排、步骤执行、运行监控、调试排障、数据输入输出和协作共享等核心能力，并确定前后端分离架构以及 REST 与 WebSocket 双通道通信方案。

在前端可视化编排方面，基于 React、Umi、TypeScript 和 ReactFlow 构建画布编辑器，支持节点拖拽、连线、参数配置、自动保存、自动布局、模板搜索、标签分组、克隆和批量删除等高频操作，为非技术用户提供低代码编排能力。

在后端执行引擎方面，基于 Kotlin 与 Ktor 提供模板管理、运行管理、调试控制、调度、认证和协作等服务；基于 Playwright Java 实现浏览器自动化执行链路，支持打开网页、点击、输入、等待、提取、文件与 Excel 操作、剪贴板读取和请求监听等节点执行。

在运行监控与调试方面，通过 WebSocket 推送运行开始、步骤开始、步骤成功、步骤失败、日志和运行结束等事件，并在调试模式下支持条件断点、时间旅行、远程操控和上下文查看，提升复杂自动化流程的可排查性。

在产品化能力方面，0.1.0 版本已形成模板市场、调度任务、认证与工作空间、协作编辑等首版闭环，使系统从单纯技术原型演进为具备初步平台属性的浏览器自动化系统。

在测试与文档方面，后端单元测试、前端构建验证、端到端回归、真实后端联调测试和文档验收均已完成，为版本收口与论文撰写提供较充分的证据支撑。

1.4 章节安排

本文共分为八章，各章内容安排如下。

第一章为绪论，主要介绍课题背景、现有方案存在的问题、课题目标和本文

主要工作。第二章介绍系统实现所涉及的相关技术与开发环境，包括 ReactFlow、Playwright、Ktor、React/umi 和 SQLite 等。第三章从可行性、业务需求和功能需求角度分析系统设计基础。第四章介绍系统概要设计，包括总体架构、模块划分、数据模型和接口设计。第五章重点展开系统详细设计，对前端编排、后端执行、通信协议、调试与协作链路进行说明。第六章说明系统实现结果与关键功能落地情况。第七章给出测试环境、测试方法、测试内容和结果分析。第八章总结全文，并对系统未来演进方向进行展望。

第 2 章 相关技术和开发环境

浏览器自动化与低代码平台的结合是近年来软件工程和业务自动化领域的一个活跃方向。浏览器自动化技术的核心目标是通过程序驱动浏览器完成页面加载、元素定位、事件触发、数据提取和结果保存等操作,替代人工完成重复性网页任务。低代码平台则将复杂的技术流程抽象为图形化配置,降低使用者的技术门槛。将二者结合,能够在兼顾灵活性的同时改善自动化系统的可用性和可维护性。

在本课题中,系统研究重点并不在于重新发明底层浏览器自动化机制,而在于如何把成熟自动化框架封装为面向普通用户的可视化 workflows 系统。因而,课题的关键技术路线包括三个方面:一是节点式可视化编排技术,解决流程建模和参数配置问题;二是浏览器自动化执行技术,解决真实页面控制和数据提取问题;三是前后端协同与实时事件推送技术,解决流程执行可观测性问题。围绕这三条技术主线,本文最终选择 ReactFlow、Playwright、Ktor 与 SQLite 作为系统实现基础。

2.1 相关技术/相关研究调研

ReactFlow 是一套面向节点式应用的前端图形库,适用于流程图编辑器、数据流编排器和低代码工作台等场景。其核心优势在于内置节点与边模型、拖拽与缩放支持、小地图、选择与多选、连线管理和布局扩展能力,能够显著降低可视化编排界面的实现成本。对于本课题而言,ReactFlow 不仅提供画布渲染能力,更为"节点+参数+连线"的流程建模提供了契合度较高的数据结构。

TheTower 以 ReactFlow 作为前端画布基础,构建了步骤库、画布区和配置面板的三栏式编辑结构。每个节点以固定类型和配置 schema 定义,用户拖拽节点到画布后,即可通过侧边栏填写 URL、选择器、文本、变量名、文件路径和监听规则等参数。借助 ReactFlow 提供的节点与边管理能力,系统可以较方便地维护主流程步骤与零散画布节点之间的关系,为模板保存、导入导出和自动布局提供了基础。

2.2 相关技术 2

Playwright 是现代浏览器自动化框架,支持 Chromium、Chrome、Firefox、WebKit 和 Edge 等多种浏览器或通道,具备智能等待、稳定元素定位、页面上下文控制和网络请求监听等能力。与早期自动化方案相比,Playwright 在现代前端页面的兼容性、等待机制和跨浏览器支持方面具有明显优势,因此非常适合作为浏览器 RPA 系统的执行内核。

TheTower 后端基于 Playwright Java 构建执行器，将前端编排的步骤树映射为浏览器操作序列。系统通过打开网页、点击、输入、等待、文本提取、属性提取、数据保存、Excel 导入导出和请求监听等执行分支，实现对真实业务页面的自动化控制。同时，Playwright 的调试能力与浏览器上下文管理能力也为系统扩展条件断点、时间旅行、远程操控和调试预览提供了技术基础。

2.3 相关技术 3

Ktor 是基于 Kotlin 的轻量级异步 Web 框架，模块化能力和协程支持较好，适合构建中小规模但结构清晰的 REST 与 WebSocket 服务。在本课题中，Ktor 承担模板管理、运行管理、调试控制、调度任务、认证与协作等接口服务职责，并与 Kotlin Coroutines 配合，为每个运行任务分配独立协程，支持启动、取消、重启和调试控制。

系统通信分为两类。第一类是 REST 接口，负责模板 CRUD、运行创建、调度维护、市场发布导入、用户登录和协作分享等具备明确请求响应边界的业务场景。第二类是 WebSocket 事件流，负责运行开始、步骤开始、步骤成功、步骤失败、日志、运行结束、调试状态和协作补丁等实时事件的推送。REST 与 WebSocket 的组合使系统既能保证管理接口的清晰性，又能提供执行过程的实时可视化能力。

2.4 开发环境

2.4.1 软件环境

本课题主要的开发环境如下表所示：

本课题开发环境以现代前后端分离项目常用工具链为基础。前端使用 React 18、Umi 4、TypeScript、MobX、Ant Design 与 ReactFlow 构建界面与状态管理；后端使用 Kotlin 2.2.20、Ktor 2.3.12、Playwright Java、Kotlinx Serialization、SQLite JDBC 和 Logback 实现服务与执行器。开发过程中，前端使用 npm 进行依赖管理与构建，后端使用 Gradle 管理依赖、测试与打包。

系统默认以本地方式部署运行。后端默认监听 8080 端口，SQLite 数据文件默认位于 backend/data/thetower.db；浏览器执行器默认使用 chromium，无头模式默认开启，步骤超时默认为 10000 毫秒。系统支持通过环境变量覆盖浏览器类型、无头模式和步骤超时，便于不同调试与测试场景切换。

本系统使用到的主要开源包如下表所示：

表 2-1 系统所需开发环境表

Ktor	2.3.12
Playwright-java	1.6.3

Jimmer	latest.release
SQLite-JDBC	3.46.1.3
React	19.2.10
xyflow/react(React-flow)	12.10.0
Ant Design	6.2.2
Axios	1.13.4
Mobx	6.15.0
Umi	4.6.25

2.4.2 硬件环境

由于系统本质上是浏览器自动化 workflow 平台，运行环境需同时承担前端界面渲染、后端服务执行和浏览器进程运行等任务，硬件环境应具备基本的多任务运行能力。开发与测试中，建议处理器为主流双核及以上配置，内存 8GB 及以上，以保证运行多个浏览器实例或并发流程时的稳定性。若需进行真实浏览器调试、长时间自动化回归或多流程并发测试，更高的处理器和内存配置会带来更稳定的体验。

第3章 系统设计

业务需求分析需明确两个核心问题：用户为何需要该系统，以及系统应服务哪些典型场景。TheTower 面向的主要是浏览器端的重复性规则任务，如后台数据录入、网页信息采集、页面监控、结果导出和接口响应观察等。这类场景通常呈现三个特征：业务流程可抽象为离散步骤；执行顺序明确且重复频率较高；运行过程中需记录状态、结果和错误信息。传统人工操作虽易于上手，但随任务频率增加，效率与准确性均会明显下降。

3.1 可行性研究

系统设计之前需要从经济、技术和运行三个角度评估课题可行性。TheTower 的设计目标并非构建面向超大规模企业场景的商业平台，而是实现一个可验证、可扩展、可演示的浏览器自动化 workflows 系统。因此，可行性分析侧重技术选型成熟度、系统复杂度可控性、项目边界与毕业设计周期的适配度，以及最终交付价值的清晰度。

3.1.1 经济可行性

本系统整体采用开源技术栈实现，前端使用 React、Umi、TypeScript 与 ReactFlow，后端使用 Kotlin、Ktor、Playwright 和 SQLite。开发、测试与部署均可在普通个人电脑上完成，无需依赖昂贵的商业授权。对于毕业设计项目而言，该技术方案能够在较低成本下获得较高的功能完整度，同时保留较好的可移植性与可扩展性。从后续推广角度看，系统更适合面向中小企业或个人用户以本地部署方式使用，因此经济上具备较高可行性。

3.1.2 技术可行性

本系统所依赖的关键技术均为成熟方案。ReactFlow 已广泛应用于节点式编辑器开发，能够支撑画布拖拽、连线和节点配置；Playwright 具备稳定的浏览器自动化能力和跨浏览器支持；Ktor 与 Kotlin 协程适合构建结构清晰、并发友好的后端服务；SQLite 则满足本课题本地持久化需求。项目文档与测试报告显示，当前系统已完成多浏览器兼容验证、50 节点画布性能验证、后端单元测试与真实后端联调，技术路线已从理论可行推进到工程可行。

3.1.3 运行可行性

从运行角度看，TheTower 的目标用户并不是专业开发人员，而是希望通过

图形化方式快速配置自动化流程的业务人员或开发者。系统通过步骤库、画布、配置面板和运行监控面板组织用户操作流程，使用上符合“拖拽节点、配置参数、连接流程、运行与调试”的直观认知。系统还补齐了自动保存、搜索、克隆、调试和协作等高频需求，降低了连续使用中的学习成本和操作成本，因此在运行层面具备较高可行性。

3.2 业务需求分析

业务需求分析需明确两个核心问题：用户为何需要该系统，以及系统应服务哪些典型场景。TheTower 面向的主要是浏览器端的重复性规则任务，如后台数据录入、网页信息采集、页面监控、结果导出和接口响应观察等。这类场景通常呈现三个特征：业务流程可抽象为离散步骤；执行顺序明确且重复频率较高；运行过程中需记录状态、结果和错误信息。传统人工操作虽易于上手，但随任务频率增加，效率与准确性均会明显下降。

从用户角度看，系统需回应五类业务需求。最基本的是低代码搭建能力——用户应能通过拖拽方式构建自动化流程，而非直接编写脚本。其次是模板复用与管理，用户需对已有模板进行搜索、分组、标签管理、克隆和删除。第三类需求围绕运行控制：用户需在执行前设置浏览器类型、超时、无头模式等参数，并在执行中实时查看进度、日志和产物。第四类需求是调试能力，帮助用户定位失败步骤和上下文问题，降低自动化流程的维护成本。此外，随着使用深入，用户还会产生模板共享、定时触发、市场复用和多人协作等产品化需求。

3.3 功能需求分析

结合项目需求文档与用户手册，可以将 TheTower 的功能需求划分为四个层面。

编排与模板管理是基础层。系统应支持新建、编辑、保存、删除、克隆、导入导出模板，并提供名称搜索、标签分组和批量删除等管理能力。编辑器方面需支持节点拖拽、连线、复制粘贴、缩放、小地图、自动布局 and 自动保存。

执行与调试是核心层。系统应同时支持普通运行和调试运行两类模式。普通运行下，系统按步骤顺序执行工作流并输出日志与结果；调试运行下，则需支持条件断点、继续执行、单步执行、上下文查看、时间旅行和预览远程操控等能力，帮助用户定位复杂问题。

数据与集成是扩展层。系统应支持文件与 Excel 节点、剪贴板节点、焦点元素节点和请求监听节点，将运行期数据写入变量或落盘保存。文件类节点需定义输入路径、输出路径和失败反馈，网络监听节点则需定义监听范围、监听生命周期和命中结果保存方式。

产品化是应用层。系统应支持模板市场发布与导入、定时调度、认证登录、工作空间切换、模板分享和协作编辑。对于 0.1.0 版本而言，这些功能已形成"前端入口+后端协议+存储链路+测试证据"的首版闭环，而非仅停留在界面入口层面。

第 4 章 概要设计

4.1 系统设计

4.1.1 系统架构设计

TheTower 采用前后端分离架构。前端承担 workflow 可视化编排、模板展示、运行监控和用户交互；后端负责模板持久化、运行管理、调试控制、调度、协作及浏览器自动化执行。两者通过 REST 接口进行标准化数据交互，借助 WebSocket 实现运行事件、调试事件和协作事件的实时同步。该架构将"界面编排"与"实际执行"解耦，前端得以专注于图形建模与交互体验，后端则可聚焦于流程编译、状态管理和自动化执行。

从执行链路看，用户先在前端画布中完成 workflow 编排，系统将节点与连线组织为流程定义后保存为模板。运行开始时，前端通过 POST /api/v1/runs 接口发送运行请求，附带模板 ID、浏览器参数和调试参数。后端 RunService 创建运行记录后，调用 PlaywrightRunExecutor 打开浏览器上下文，并在 PlaywrightRunSession 中逐步执行步骤树。执行过程中，系统持续向 WebSocket 通道推送 RUN_STARTED、STEP_STARTED、STEP_SUCCEEDED、STEP_FAILED、LOG、RUN_SUCCEEDED 等事件，使前端能够实时呈现执行过程。

4.1.2 系统功能模块

从功能划分看，系统由六个模块构成。模板管理模块负责模板创建、编辑、查询、更新、删除、克隆、批量删除以及模板市场发布和导入。流程编排模块负责节点库展示、节点拖拽、连线关系维护、参数配置和自动布局。运行与调试模块负责普通运行、调试运行、断点控制、时间旅行、预览远程操控和日志查看。数据与集成模块负责文件、Excel、剪贴板、焦点元素和请求监听节点的执行与结果处理。平台能力模块负责调度任务、登录认证、工作空间与协作编辑。协议与存储模块负责 REST/WS 协议、SQLite 数据持久化、运行产物与协作数据落盘。

各模块之间并非孤立工作，而是通过统一数据模型和协议相互协同。例如，模板管理模块维护 WorkflowTemplate，流程编排模块修改模板中的 steps 与

otherStep，运行与调试模块消费模板数据生成 Run 记录，平台能力模块则通过模板和工作空间建立访问边界。这样的模块划分既有利于前后端职责清晰，也有利于后续扩展新节点和新能力。

4.2 系统核心功能设计

4.2.1 关键概念定义

系统涉及的核心概念包括模板、步骤、运行、调试会话、工作空间和协作会话。模板是工作流的持久化载体，描述某一自动化流程的结构、元数据和最近运行摘要。步骤是模板中的最小可执行单元，每个步骤包含 type 与 config 两部分，分别表示节点类型和参数配置。运行是某次模板执行的实例，记录运行状态、当前步骤、起止时间和错误信息。调试会话是运行的特殊态，支持暂停、继续、单步和远程操控等行为。工作空间用于隔离不同用户或团队的模板资源，协作会话则管理模板的共享、在线成员状态和补丁同步。

4.2.2 模型建立

系统接口分为模板接口、运行与调试接口、模板市场接口、调度接口、认证与工作空间接口以及协作接口六大类。模板接口包括 GET /templates、POST /templates、PUT /templates/{id}、PATCH /templates/{id}、POST /templates/{id}/clone 和 POST /templates/batch-delete 等，用于完成模板的生命周期管理。运行接口包括 POST /runs、GET /runs、GET /runs/{id}、POST /runs/{id}/cancel 和 POST /runs/{id}/restart 等。调试接口则扩展了 open-browser、continue、step、close 和 remote-control 等行为，支持调试态下的浏览器控制。

在事件模型方面，系统定义了运行事件、调试事件和协作事件三类消息。运行事件包括 RUN_STARTED、STEP_STARTED、STEP_SUCCEEDED、STEP_FAILED、LOG、RUN_SUCCEEDED、RUN_FAILED 和 RUN_CANCELED；调试事件包括 DEBUG_SESSION_STARTED、DEBUG_FRAME、DEBUG_BREAKPOINT_HIT、DEBUG_CONTEXT_UPDATED、DEBUG_STEPPED 等；协作事件包括在线成员变化和补丁应用通知。统一的事件模型使前端可以通过同一事件管道感知不同执行阶段的系统状态。

4.3 系统接口设计

平台能力接口中，模板市场接口支持发布与导入；调度接口支持单次任务与周期任务配置；认证接口支持登录、登出、当前用户信息与工作空间列表查询；协作接口支持模板分享、presence 心跳和在线成员快照。所有 REST 接口统一

采用 `/api/v1` 作为前缀，响应采用 `requestId`、`data` 与 `error` 的包装格式。WebSocket 方面，运行事件通道为 `/ws/v1/runs/{runId}`，协作事件通道为 `/ws/v1/templates/{templateId}/collaboration`。统一协议设计有利于前后端联调和测试资产复用。

平台能力接口中，模板市场接口支持发布与导入；调度接口支持单次任务与周期任务配置；认证接口支持登录、登出、当前用户信息与工作空间列表查询；协作接口支持模板分享、`presence` 心跳和在线成员快照。所有 REST 接口统一采用 `/api/v1` 作为前缀，响应采用 `requestId`、`data` 与 `error` 的包装格式。WebSocket 方面，运行事件通道为 `/ws/v1/runs/{runId}`，协作事件通道为 `/ws/v1/templates/{templateId}/collaboration`。这种统一协议设计有利于前后端联调和测试资产复用。

第 5 章 详细设计

5.1 流程编排模型与前端交互设计

前端编排模块以 ReactFlow 为核心, 围绕"步骤库、画布、配置面板"三个区域组织用户交互。步骤库展示可用节点类型, 画布承载节点的放置、拖拽、连线 and 布局, 配置面板负责节点参数和运行参数的输入。系统通过 stepRegistry 维护节点注册表, 每个节点具有固定的 type、label、group、icon 和表单 schema, 保证节点配置的一致性和可扩展性。当前系统支持的节点类型已覆盖页面操作、等待、数据提取、文件与 Excel 处理、剪贴板、焦点元素和网络监听等多种类别。

在画布交互方面, 系统除基础拖拽和连线外, 还支持复制粘贴、撤销重做、缩放、小地图和自动布局。0.1.0 版本补齐了自动保存机制, 使编辑结果能够在一定周期内自动写回后端, 同时提供状态提示避免用户误判保存结果。首页模板管理也增加了搜索、标签分组、克隆和批量删除等能力, 使模板数量增长后仍然便于维护。前端对这些能力的组织方式, 体现了系统从“可编辑”向“可长期使用”转变的设计思路。

5.2 后端执行引擎设计与实现

根据测试报告, 后端验证包括模板体验增强接口、运行参数与调试协议、数据文件与网络监听节点、调度任务接口、模板市场接口、认证与工作空间、协作与在线状态等七类核心用例, 均已通过。相关证据包括 TemplateServiceTest、DebugExecutionGateTest、PlaywrightRunExecutorTest、StepTreeSupportTest、SchedulerServiceTest、TemplateMarketServiceTest、AuthServiceTest、TemplateCollaborationServiceTest 和 TemplatePresenceServiceTest 等测试资产。

执行开始后, 系统首先发送 RUN_STARTED 事件, 随后按照 steps 数组顺序逐步执行节点。每个步骤执行前发送 STEP_STARTED 事件, 执行成功后发送 STEP_SUCCEEDED 事件, 若失败则发送 STEP_FAILED 事件并附带错误码与错误信息。对于需要产物输出的节点, 例如 saveData、saveExcel 和 importExcel, 系统会将结果写入运行数据目录或变量上下文, 供后续步骤继续使用。对于 listenRequestTrigger 和 listenRequestResult 等节点, 系统会在浏览器会话中注册监听器, 并在命中请求后把结果写入变量, 从而让网络事件可以被后续步骤消费。

该执行器设计的关键在于将"抽象节点"映射为"具体浏览器操作"。例如, openUrl 映射为页面打开, click 映射为元素点击, type 映射为定位并输入文本, extract 则根据 mode 决定提取 innerText 或 attribute。通过统一节点类型和参数

结构,系统能够在不暴露 Playwright 细节的前提下提供较稳定的自动化能力——这也是本课题区别于直接编写脚本的核心设计选择。

5.3 调试机制与运行时上下文设计

复杂自动化流程在真实页面中运行时,常会受到元素变化、网络延迟、变量值异常和步骤配置错误等因素影响,因此调试能力是系统可维护性的关键。TheTower 在普通运行之外提供调试运行模式,并设计了以 `DebugExecutionGate` 和调试事件流为核心的调试机制。调试参数中包含是否启用调试、是否打开可见浏览器、是否打开 `DevTools`、预览帧率、预览质量、是否启动后暂停、断点列表和结束后是否保留浏览器等字段,保证普通运行和调试运行在协议语义上保持一致。

在调试运行过程中,系统可根据条件断点或命中次数条件暂停执行,并向前端发送 `DEBUG_BREAKPOINT_HIT`、`DEBUG_CONTEXT_UPDATED` 和 `DEBUG_FRAME` 等事件。前端调试工作台据此展示当前步骤、变量上下文、预览画面和运行轨迹,用户可选择继续执行、单步执行或通过 `remote-control` 接口在调试预览中发送点击、输入等指令。时间旅行功能用于回看关键步骤、变量快照和运行轨迹的历史状态,帮助用户快速理解失败前后的执行过程。

运行时上下文不仅包含变量和值,还包含页面会话、产物路径、调试状态和监听器注册等信息。执行某些节点时,系统会将提取结果或请求监听结果写入变量空间,供后续条件分支、循环或保存节点使用。由此,系统形成了从页面操作、数据提取到条件决策的闭环,自动化流程的表达能因此有所提升。

5.4 协议、协作与数据持久化设计

TheTower 的协议设计遵循"REST 管理+WebSocket 事件"的双轨方案。REST 处理请求响应边界明确的操作,如创建模板、查询运行列表、创建调度、登录和分享模板;WebSocket 则负责推送运行状态、调试信息和协作变更。这种设计简化了前端状态管理,同时增强了执行过程的实时性。对于浏览器自动化系统而言,单纯依赖轮询难以低延迟展示步骤级别状态,而事件流机制能更自然地支撑运行监控与调试工作台。

协作设计方面,系统增加了模板分享、在线成员快照、`presence` 心跳、协作事件流和差异采纳能力。用户在模板卡片中可配置分享对象与权限,进入编辑后可查看在线成员和远端保存提示,当出现冲突时可查看差异摘要、差异详情,并选择性采纳节点或连线变更。当前协作能力已形成首版闭环,但复杂多方并发场景下的细粒度自动合并仍属于后续优化方向。

数据持久化方面,系统主要使用 `SQLite` 保存核心业务数据,并把运行产物、

协作数据等以文件方式落盘。SQLite 满足本课题本地单体部署、快速开发和低运维成本的目标；文件持久化则适合存放运行日志、导出数据和协作补丁等附属内容。这样的设计使系统能够在毕业设计规模下保持实现简洁，同时为后续向更强持久化与审计能力演进预留空间。

第 6 章 系统实现

系统首页已实现模板创建、编辑、删除、导入导出、搜索、标签分组、克隆和批量删除等能力。用户进入首页后可通过“新建模板”创建 workflow 模板，也可使用关键字搜索和标签分组对模板进行筛选。对于已有模板，可通过克隆快速生成副本，避免重复搭建基础流程。批量删除功能用于集中清理无效模板，提高管理效率。上述能力在 0.1.0 版本中已完成前后端接口与回归测试闭环。

6.1 模板管理与编辑器实现

系统首页已实现模板创建、编辑、删除、导入导出、搜索、标签分组、克隆和批量删除等能力。用户进入首页后可通过“新建模板”创建 workflow 模板，也可使用关键字搜索和标签分组对模板进行筛选。对于已有模板，可通过克隆快速生成副本，避免重复搭建基础流程。批量删除功能用于集中清理无效模板，提高管理效率。上述能力在 0.1.0 版本中已完成前后端接口与回归测试闭环。

编辑器页面采用三栏式布局。左侧为步骤库，中间为 ReactFlow 画布，右侧为节点配置与运行参数面板。用户完成拖拽、连线和参数填写后，系统会通过自动保存机制持续同步编辑内容，从而降低手动保存遗漏风险。编辑器还提供自动布局能力，能够在主流程和基础分支场景下快速整理节点位置，使画布结构更加清晰。这些体验增强功能虽然不直接参与浏览器执行，但决定了系统是否适合长期使用，因此在本版本中被作为重要实现项完成。

6.2 执行与调试功能实现

在执行层面，系统实现了普通运行与调试运行双模式。普通运行模式下，用户可以在运行前设置浏览器类型、headless 和默认超时等参数，随后发起运行并查看实时状态、日志和产物。调试运行模式下，用户可进一步启用可见浏览器、DevTools、断点列表、预览帧率和暂停启动等参数。执行器会根据这些参数创建对应浏览器环境，并在必要时暂停流程等待用户操作。

调试工作台已形成可用闭环。用户可通过继续执行、单步执行、上下文查看和远程操控等操作定位问题。系统支持条件断点、时间旅行和调试预览首版能力，能够在失败场景中更准确地定位步骤、变量与页面状态变化。相较于只给出“执行失败”这种粗粒度反馈，TheTower 把失败步骤、错误信息、上下文变量和调试操作统一纳入一套工作台中，问题排查效率因此有明显提升。

6.3 数据与网络监听节点实现

在节点能力扩展方面，系统实现了 saveData、saveExcel、importExcel、

getClipboardText、extractActiveElement、listenRequestTrigger、listenRequestResult 和 stopPageListen 等节点。saveData 和 saveExcel 用于将运行期结构化数据输出为文件，importExcel 用于读取外部 Excel 数据参与执行；getClipboardText 可用于读取运行设备上的剪贴板内容，extractActiveElement 则可在页面交互场景中获取当前焦点元素信息。网络监听节点支持监听请求触发或响应结果，并将命中结果写入变量，供后续节点复用。

这些节点的实现使系统的应用场景不再局限于页面点击和文本输入，而是能够覆盖更完整的数据输入输出与运行时观测。例如，用户可以先导入 Excel 数据，再循环执行页面录入任务；也可以在页面请求触发时截取响应结果并保存文件。这类能力让系统更贴近真实业务自动化场景，而非仅停留在教学演示层面。

从功能结果看，系统在 0.1.0 版本中已完成体验增强、高级调试、数据文件与网络监听节点、模板市场、调度任务、认证与工作空间、协作编辑等首版闭环。项目不仅实现了 workflow 编排与浏览器执行，还补齐了运行监控、问题排查、模板复用和多人协作等实际使用场景中必需的能力。从测试结果看，系统已完成后端单元测试、前端构建验证、回归测试、真实联调测试和文档验收，表明项目已从“可运行原型”提升到“可验证交付”的阶段。

为了让系统具备产品化首版能力，0.1.0 版本补齐了模板市场、调度任务、认证与工作空间以及协作编辑功能。模板市场支持浏览、发布和导入模板，降低了模板复用成本。调度任务支持单次任务和间隔任务，用户可在首页创建调度并查看最近触发结果。认证与工作空间功能用于区分不同用户和不同工作空间下的资源访问边界，使系统不再局限于单用户本地工具。

协作编辑是本版本的重要增强点。系统支持模板分享、在线成员展示、presence 心跳、远端保存同步、冲突提示和差异采纳。用户可以将模板分享给其他协作者，并在编辑器中实时感知在线成员和远端更新。当本地修改与远端修改冲突时，系统能够展示差异摘要和节点级差异内容，帮助用户做出选择。虽然当前协作能力仍属于首版实现，但它已让系统具备了明显的平台化特征。

第 7 章 系统测试

7.1 测试环境和测试方法

系统测试覆盖后端测试、前端验证、端到端验证和文档交付验收四个维度。后端侧主要通过 Gradle 测试和服务级测试类验证模板服务、运行服务、调试执行门、市场、调度、认证与协作等核心逻辑；前端侧主要通过构建验证和 Playwright E2E 用例验证编辑器、模板管理、调试 UI 和平台入口；端到端测试侧通过真实后端联调和数据文件、网络监听节点回归验证完整闭环；文档验收则根据用户手册、API 文档和部署文档复现操作链路，确保项目交付材料与实际系统一致。

从测试方法看，系统综合采用单元测试、集成测试、回归测试、端到端测试和文档复现测试。单元测试用于验证关键类的逻辑正确性，例如执行器浏览器适配、模板服务和调试门控逻辑；集成与回归测试用于验证模板管理、运行调试和平台能力入口是否稳定；端到端测试用于验证真实浏览器和真实后端环境下的可用性；文档测试则用于检验交付材料是否真实可复现。多层次测试方式保证了系统不仅“功能存在”，而且“能够持续工作”。

7.2 测试用例设计和测试方案

根据测试报告，后端验证包括模板体验增强接口、运行参数与调试协议、数据文件与网络监听节点、调度任务接口、模板市场接口、认证与工作空间、协作与在线状态等七类核心用例，均已通过。相关证据包括 TemplateServiceTest、DebugExecutionGateTest、PlaywrightRunExecutorTest、StepTreeSupportTest、SchedulerServiceTest、TemplateMarketServiceTest、AuthServiceTest、TemplateCollaborationServiceTest 和 TemplatePresenceServiceTest 等测试资产。

前端验证包括自动保存与体验增强、搜索标签克隆批量删除、运行参数面板、高级调试 UI、模板市场入口、调度任务入口以及账户权限与协作入口等七类用例。端到端验证则围绕体验增强闭环、高级调试闭环、数据文件与网络监听闭环、产品化能力闭环四大场景展开。除此之外，系统还保留了 backend/scripts/api-smoke-test.ps1 和 backend/scripts/ws-event-test.ps1 两类脚本，用于在本地快速验证 REST 与 WebSocket 链路。

针对性能与兼容性，系统重点围绕 50 节点画布性能和多浏览器执行稳定性设计测试方案。前端性能测试关注画布渲染时间、节点选择响应和拖拽流畅度；后端兼容性测试关注 Chromium、Firefox、WebKit 及 Chrome、Edge 通道的执行适配。由于浏览器 RPA 系统通常要面对真实页面环境和长时间执行问题，因

此这些测试内容对毕业设计成果的可用性评价具有较高价值。

7.3 结果和分析

测试结果显示，0.1.0 版本已形成“实现证据+回归证据+文档证据”的完整闭环。后端 `gradle test` 与前端 `npm run build` 均已执行并通过；模板管理、运行调试、数据文件节点、网络监听节点、市场、调度、认证与协作主线均已通过验证；历史缺陷如模板市场兼容、调度 404、协作冲突提示缺失和真实联调启动抖动等问题也已完成修复与回归。

从性能结果看，系统已完成 50 节点画布性能测量与回归资产沉淀，画布渲染、节点选择与拖拽在目标规模内保持稳定。需求文档中提出的“支持至少 50 个节点流畅拖拽与渲染”和后端“支持 Chrome、Firefox、Edge 三款浏览器自动化、单流程执行成功率大于等于 95%、支持并发执行至少 3 个流程”等指标，已通过现有验证基线获得较充分支撑。测试结果表明，TheTower 在功能上达成了毕业设计目标，在工程稳定性和产品闭环方面也具备较好的基础。

第 8 章 总结与展望

8.1 总结

本文围绕“降低浏览器自动化门槛”这一目标，设计并实现了一个基于 Playwright 的浏览器 RPA 画布 workflow 系统 TheTower。系统采用前后端分离架构，前端通过 ReactFlow 提供低代码可视化编排能力，后端通过 Kotlin、Ktor 和 Playwright 提供执行、调试、调度、认证和协作等核心能力，并通过 REST 和 WebSocket 实现管理操作与实时状态反馈。相比传统脚本式自动化方案，TheTower 更侧重可视化、可观测性与可复用性；相比重量级商业 RPA 平台，TheTower 更侧重轻量、低成本与本地可部署。

从功能结果看，系统在 0.1.0 版本中已完成体验增强、高级调试、数据文件与网络监听节点、模板市场、调度任务、认证与工作空间、协作编辑等首版闭环。项目不仅实现了 workflow 编排与浏览器执行，还补齐了运行监控、问题排查、模板复用和多人协作等实际使用场景中必需的能力。从测试结果看，系统已完成后端单元测试、前端构建验证、回归测试、真实联调测试和文档验收，表明项目已从“可运行原型”提升到“可验证交付”的阶段。

从毕业设计意义看，本课题的价值主要体现在两个方面。其一，系统以工程方式验证了 ReactFlow 与 Playwright 结合构建低代码浏览器自动化平台的可行性。其二，系统围绕实际使用路径补齐了调试、数据节点和平台能力，使其不再只是技术展示，而具有一定的产品雏形。这些成果说明，浏览器自动化系统完全可以在保持执行能力的同时，以更友好的方式服务普通用户。

8.2 展望

TheTower 的 0.1.0 版本已能跑通完整的浏览器自动化流程，但若用于多人协作的生产环境，仍有若干不足。协作方面，目前支持模板分享、在线成员查看和差异采纳，但多人同时编辑同一复杂流程时，自动合并策略尚显薄弱——部分不冲突的改动也会触发人工介入提示。后续应细化到操作级别的补丁追踪，按冲突类型分别处理，同时增强状态机的可视化，让用户更直观地理解冲突来源。权限方面，当前仅能控制工作空间和模板分享，粒度较粗。实际团队中，不同成员往往需要不同的操作权限——有的仅可查看，有的可编辑但不可删除，敏感操作还需留存审计日志。后续应支持更细粒度的操作授权与角色模型，并建立完整的操作审计机制，以适应更复杂的团队协作场景。

调试功能方面，条件断点、时间旅行、远程操控等能力在 0.1.0 版本中已可用，但覆盖面仍有限。复杂业务流程往往涉及数十个步骤、多重分支判断和各类

异常恢复，这类场景的端到端自动化验证尚不充分。后续需针对长流程、复杂分支和异常恢复等场景补充系统性的自动化测试。数据存储方面，当前采用 SQLite 加本地文件，对于毕业设计和轻量部署场景已足够。但若面向生产环境处理更大规模的业务，该方案在持久化能力和部署弹性上存在瓶颈。后续需引入更专业的数据库与对象存储，并建立运维审计体系，使持久化能力与部署弹性有所提升。总体来看，TheTower 已为浏览器 RPA 低代码化探索提供了完整的工程样本。后续若沿协作增强、权限细化、调试强化和持久化升级方向持续演进，系统有望在教学研究之外形成更强的实际应用价值。

总体来看，TheTower 已为浏览器 RPA 低代码化探索提供了完整的工程样本。后续若沿协作增强、权限细化、调试强化和持久化升级方向持续演进，系统有望在教学研究之外形成更强的实际应用价值。

参考文献

- [1] React Flow Documentation Team. React Flow Documentation: Learn how to build node-based applications[EB/OL]. <https://reactflow.dev/learn>.
- [2] Microsoft. Playwright Documentation: Reliable end-to-end testing for modern web apps[EB/OL]. <https://playwright.dev/docs/intro>.
- [3] JetBrains. Ktor Documentation: Build asynchronous servers and clients in Kotlin[EB/OL]. <https://ktor.io/docs/welcome.html>.
- [4] 徐小夕. React-Flow 中文文档正式上线: 指南 [EB/OL]. <https://www.cnblogs.com/gccbuaa/p/19205817>.
- [5] ycfenxi. ReactFlow: 构建交互式节点流程图的完全指南 [EB/OL]. <https://www.cnblogs.com/ycfenxi/p/19115406>.
- [6] 霍格沃兹测试学社. Playwright UI 自动化实战速成指南 [EB/OL]. <https://bbs.huaweicloud.com/blogs/462395>.
- [7] TheTower 项目文档组. TheTower API 文档 (0.1.0) [Z]. 项目文档.
- [8] TheTower 项目文档组. TheTower 用户使用手册 (0.1.0) [Z]. 项目文档.
- [9] TheTower 项目文档组. TheTower 0.1.0 技术实现报告[Z]. 项目文档.
- [10] TheTower 项目文档组. TheTower 0.1.0 测试报告[Z]. 项目文档.

致谢

本毕业设计的完成离不开多方的支持与帮助。首先要感谢指导老师。从选题阶段开始，老师便引导我思考真正感兴趣且值得深入的方向，而非随意选择一个题目交差。系统架构设计初期，我曾一度追求大而全的方案，老师一句“先让最小闭环跑起来”让我少走了不少弯路。论文撰写过程中，老师逐章审阅并给出具体意见——哪些地方需要展开论证，哪些只是功能点的罗列，都有明确指出。能在毕业设计周期内从需求分析走到测试收口，离不开这些持续的指导。同时，也感谢在项目开发和学习过程中给予我帮助的同学与朋友。**ReactFlow** 的节点拖拽逻辑曾让我卡了较长时间，是同班同学帮忙排查代码后才发现是事件委托的问题；后端 **WebSocket** 推送的状态同步一度出现数据混乱，也是朋友提醒我应加上消息序号。测试反馈和资料整理方面，我和同学们也是互相帮忙。没有这些协助，课题推进的效率会低很多。最后，感谢学校和学院提供的学习环境与实践平台。图书馆的资料库、学院定期组织的毕设进度汇报，这些资源和机制为毕业设计的顺利推进提供了保障。毕业设计的完成也标志着大学阶段学习生活的阶段性收尾。这段经历让我认识到，完成一个完整的系统远比编写几段能运行的代码困难，但也正因为如此，过程中的收获才更为扎实。

同时，也感谢在项目开发和学习过程中给予我帮助的同学与朋友。无论是在技术讨论、测试反馈还是资料整理方面，这些帮助都使我能够更高效地推进课题研究。最后，感谢学校和学院提供的学习环境与实践平台，使本次毕业设计得以顺利完成，并为大学阶段的学习生活画上阶段性句号。